

♦ JONGO 0.1

How Jongo works, and why it beats Django

A full-stack Python web framework where the pages, the server code and the interactive frontend are all written in one language — and the frontend compiles to JavaScript.

Written for Joshua Harty · 17 September 2026

Source: `~/jongo` · 6,662 lines of Python · 1,476 lines of browser runtime · 117 tests · zero dependencies

CONTENTS

What's in this guide

Read it beside the code in VS Code. Every section names the files it describes.

- | | | | |
|---|------------------------------------|----|------------------------------------|
| 1 | The idea in one page | 7 | Models and migrations |
| 2 | A whole app, annotated | 8 | Batteries: auth, admin, dev server |
| 3 | What happens when a page loads | 9 | Jongo vs Django, honestly |
| 4 | The compiler: Python to JavaScript | 10 | By the numbers |
| 5 | The browser runtime | 11 | Limits and what's next |
| 6 | Server functions | 12 | Cheat sheet and file map |

The one-sentence version

Django gives you a great backend and then leaves the browser to you. Jongo keeps the backend — an ORM, migrations, auth, sessions, CSRF, an admin site — and adds the missing half: you write your interface as Python functions, Jongo renders them on the server for the first paint, compiles the same functions to JavaScript, and wakes them up in the browser so they react to clicks without a page reload.

The test that matters. A todo list with live updates, written in Django, is `models.py` + `urls.py` + `views.py` + `forms.py` + a template + an API endpoint + a JavaScript file. In Jongo it is one file of about 60 lines, and you never write a fetch call, a template tag, or a serializer.

The idea in one page

One feature should live in one place.

In Django a single feature is scattered by file type rather than by purpose. The URL lives in one file, the view in another, the markup in a third, and anything interactive in a fourth language entirely. You hold the whole map in your head to change one button.

CONCERN	DJANGO	JONGO
URL	<code>urls.py</code> , a separate <code>path()</code> entry	<code>@app.page("/todos/<id>")</code> on the function
View	<code>views.py</code>	the decorated function itself
Markup	a <code>.html</code> template in a second language	Python calls: <code>div(h1(title))</code>
Interactivity	hand-written JavaScript, or HTMX	the same component, compiled to JS
AJAX endpoint	view + URL + <code>fetch()</code> + JSON handling	<code>@server</code> function, called with <code>await</code>
Input validation	form or serializer classes	type hints: <code>title: str</code> , <code>todo: Todo</code>
Schema changes	<code>makemigrations</code> + migration files in git	<code>jongo migrate</code> diffs models against the live DB
Static assets	<code>collectstatic</code> , a bundler for JS	one generated file, no build step

The same feature, both ways

DJANGO — FOUR FILES

```
# urls.py
path("todos/", views.todo_list,
name="todos")

# views.py
def todo_list(request):
    todos = Todo.objects.all()
    return render(request, "todos.html",
        {"todos": todos})

# todos.html
{% for todo in todos %}
    <li>{{ todo.title }}</li>
{% endfor %}

# todos.js — and an API view for it
fetch("/api/todos/", {method: "POST",
    headers: {"X-CSRFToken": token},
    body: JSON.stringify({title})})
    .then(r => r.json())
    .then(addRow)
```

JONGO — ONE FILE

```
@server
def add_todo(title: str) -> dict:
    return
    Todo.create(title=title).to_dict()

@Component
def TodoList(todos):
    items = state(todos)

    async def add(event):
        todo = await add_todo(draft.value)
        items.value = items.value + [todo]

    return ul(
        [li(t["title"], key=t["id"])
         for t in items.value],
        on_click=add,
    )

@app.page("/todos")
def todos():
    return TodoList(todos=Todo.all())
```

No template language to learn, no serializer, no CSRF plumbing, no fetch call, no build tooling. The `await add_todo(...)` on the right is a real HTTP round trip that Jongo generates for you.

A whole app, annotated

This is the example in `examples/todo/app.py`, trimmed. Nothing is hidden: there is no other file.

```

from jongo import Jongo, component, db, effect, js, server, state
from jongo.html import *

app = Jongo(__name__, title="Jongo Todos", database="todos.sqlite3")
app.admin()                                # generated admin at /admin

class Todo(db.Model):                      # ① a table
    title = db.Text(max_length=200)
    done = db.Bool(default=False)
    created = db.DateTime(auto_now_add=True)

@server                                  # ② server code the browser may call
def toggle_todo(todo: Todo) -> dict:      # `todo: Todo` loads the row by id
    todo.done = not todo.done
    todo.save()
    return todo.to_dict()

@component                              # ③ UI: Python now, JavaScript in the browser
def TodoApp(initial):
    todos = state(initial)                  # reactive state
    show = state("all")

    remaining = len([t for t in todos.value if not t["done"]])

    def update_title():                     # ④ runs in the browser after render
        js.document.title = f"{remaining} left"

    effect(update_title, [remaining])

    async def toggle(todo):                 # ⑤ an event handler that calls the server
        updated = await toggle_todo(todo["id"])
        todos.value = [updated if t["id"] == todo["id"] else t for t in todos.value]

    visible = [t for t in todos.value if show.value == "all" or (show.value == "done") ==
t["done"]]

    return section(                         # ⑥ elements are just function calls
        ul([TodoItem(todo=t, on_toggle=toggle, key=t["id"]) for t in visible]),
        footer(span(f"{remaining} left")),
        class_=styles.card,
    )

@app.page("/")                           # ⑦ route, view and page in one
def home():
    return TodoApp(initial=Todo.all())

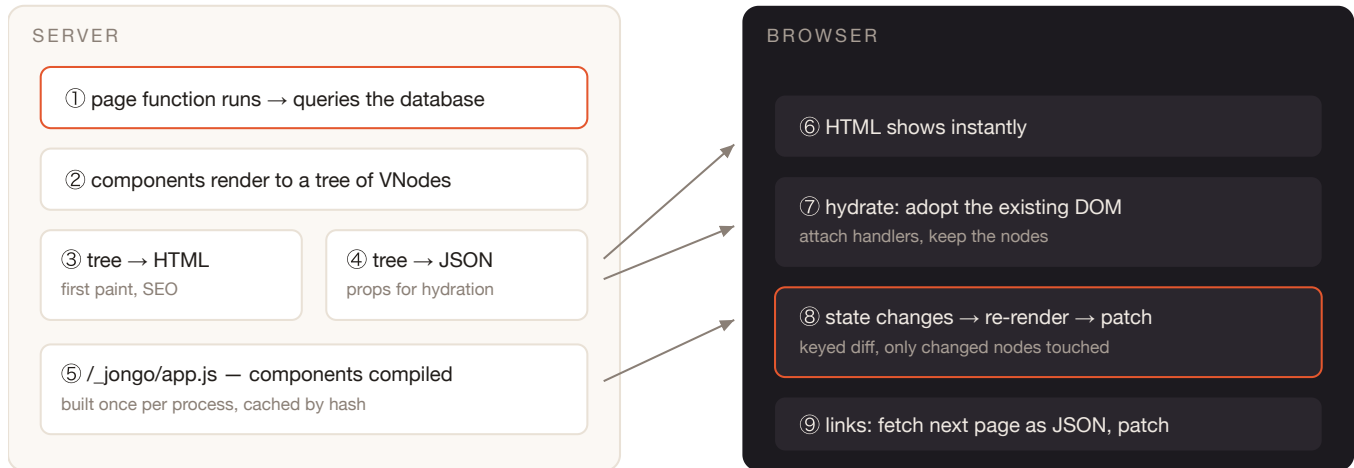
```

WHAT IT GIVES YOU

- ① A SQLite table. `jongo migrate` creates or alters it by comparing this class to the database — no migration files.
- ② An HTTP endpoint that only this decorator exposes. Arguments are checked against the hints; `todo: Todo` fetches row `id` or returns 404.
- ③ A component: rendered to HTML on the server for the first paint, compiled to JavaScript for everything after.
- ④ `effect()` is browser-only. It re-runs when `remaining` changes, and can return a cleanup function.
- ⑤ A real async function in the browser. The `await` is a POST with a CSRF token, JSON encoding and error handling that you never write.
- ⑥ Markup as Python: `class_`, `style={...}` dicts, `on_click` handlers, and `key=` for efficient list updates.
- ⑦ The page function runs on the server, queries the database directly, and passes plain data to the component.

What happens when a page loads

Server-rendered HTML first, then the same tree wakes up in the browser. Files: `jongo/app.py`, `jongo/vdom.py`, `jongo/compiler/dom.js`.



One tree, two renderers. `jongo/vdom.py` renders it to HTML on the server; `dom.js` renders the same shape to DOM nodes in the browser, so the two can meet in the middle.

Why hydration instead of a single-page app

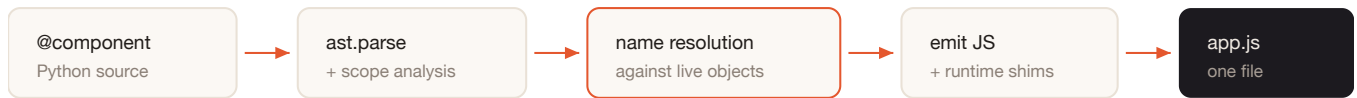
The browser receives finished HTML, so the first paint does not wait for JavaScript, and search engines and link previews see real content. The JSON copy of the tree arrives in the same document in a `<script type="application/json">` tag, so the browser rebuilds the component tree with exactly the data the server used — no second round trip, no API to write.

Navigation keeps the page alive

Clicking an internal link does not reload the page. The router fetches the target with an `X-Jongo-Nav` header, gets the tree as JSON, and patches the DOM. Your layout — and any state inside it — survives. A verified example from this build: clicking "details" moved from `/` to `/todos/2`, changed the title, and kept the same document alive.

The compiler: Python to JavaScript

The interesting part. Files: `jongo/compiler/transpile.py` (745 lines), `scope.py`, `bundle.py`.



The trick: the compiler reads live Python objects, not just text

A transpiler that only sees source text cannot know what `add_todo` is. Jongo compiles from the real function object, so it can look the name up in that function's own globals and closure and see the actual value. That single decision is what makes the API feel seamless:

WHAT THE NAME TURNS OUT TO BE	WHAT THE COMPILER EMITS
a <code>@server</code> function	an RPC stub that POSTs to <code>/_jongo/rpc/<id></code>
another <code>@component</code>	that component, compiled too (recursively)
a plain helper function in your project	the function, compiled
an HTML tag like <code>div</code>	the element factory from the runtime
a stylesheet from <code>css(...)</code>	the generated class names, inlined as data
a constant (list, dict, number)	the value, inlined as JSON
a database model	a compile error — with the advice to load it in the page function or a <code>@server</code> function
a stdlib module that can't run in a browser	a compile error naming the module and suggesting <code>@server</code> or <code>js.*</code>

Real input, real output

This is genuine output from `jongo/compiler`, not an illustration:

YOU WRITE

```

@Component
def Greeting(name, greeting="Hello"):
    clicks = state(0)

    async def click(event):
        await save(name)
        clicks.value += 1

    return div(
        h1(f"{greeting}, {name}!"),
        button(f"Clicked {clicks.value}
times",
            on_click=click),
  
```

JONGO GENERATES

```

// the @server function became a stub
const rpc__save = $rpc("sample.save",
false);

const Greeting =
$component("sample.Greeting",
  $fn(function Greeting(name, greeting =
"Hello") {
    let click, clicks;
    clicks = $state(0);
    click = $fn(async function click(event) {
        (await rpc__save(name));
        const $t1 = clicks;
  
```

```

    ul([li(word.upper())
        for word in name.split() if
word]),
    class_="card",
    )

```

```

    $t1.value = $iadd($t1.value, 1);
  }, ["event"]);
  return $call($h("div"), [
    $h("h1")(`${$str(greeting)}`,
    `${$str(name)}!`),
    $call($h("button"),
      [`Clicked ${$str(clicks.value)}
times`],
      {"on_click": click}),
    $h("ul")((( $i => { const $r = [];
      for (const word of $iter($i)) {
        if (!$t(word)) continue;
        $r.push($h("li")($m(word, "upper",
[])));
      } return $r; } )($m(name, "split",
[])))
    ], {"class_": "card"});
  }, ["name", "greeting"]));

```

Python semantics, kept

Those `$` helpers exist because JavaScript disagrees with Python in ways that would otherwise cause silent bugs. `pyrt.js` restores the Python behaviour:

EXPRESSION	PLAIN JAVASCRIPT	PYTHON	JONGO
<code>if []:</code>	truthy 😊	falsy	falsy
<code>[1] + [2]</code>	"12" 😊	<code>[1, 2]</code>	<code>[1, 2]</code>
<code>xs[-1]</code>	undefined	last item	last item
<code>-7 // 2</code>	-3.5	-4	-4
<code>{"a":1} == {"a":1}</code>	false	True	True
<code>d["missing"]</code>	undefined	KeyError	KeyError
<code>", ".join(xs)</code>	no such method	works	works
<code>f"{x:,.2f}"</code>	no format specs	works	works

This is enforced, not hoped for: `tests/test_compiler.py` runs a suite of sample functions in Python and in Node and fails unless the results are identical, covering arithmetic, strings, lists, dicts, comprehensions, exceptions, closures and calling conventions.

One deliberate improvement on Python. In Python, closures made in a loop all see the last value — the classic `lambda: i` bug. Because event handlers in loops are the single most common place this bites, Jongo compiles loop variables to per-iteration bindings when they're only used inside the loop. `button(on_click=lambda e: remove(todo))` inside a `for` loop does what you meant. There's a test that pins this behaviour.

What doesn't compile

Classes, `with` blocks, imports inside functions, generators and the parts of the standard library that assume a filesystem. You get a compile error at startup, naming the file and line, with a hint — not a mystery at runtime. Everything else stays on the server, where it belongs.

The browser runtime

1,476 lines, no dependencies, no build step. Files: `pyrt.js` (Python semantics, 780 lines) and `dom.js` (UI, 696 lines).

PIECE	WHAT IT DOES
Virtual DOM	The same node shape the server uses. Elements, text and components in one tree.
Hydration	Walks the server's HTML and adopts it instead of rebuilding: no flash, no double render. Mismatches fall back to replacing that node and warn in dev.
Keyed diffing	Reorders existing DOM nodes by <code>key=</code> , so inputs keep focus and rows aren't recreated.
Hooks	<code>state</code> , <code>effect</code> , <code>ref</code> — per component instance, with cleanup on unmount.
Scheduling	State changes mark components dirty and re-render once per microtask, parents before children, so ten updates in one handler cost one render.
Events	One listener per element per event type, swapped in place. Submit handlers get <code>preventDefault()</code> automatically.
Router	Intercepts same-origin links, fetches the page as JSON, patches, handles Back/Forward, and falls back to a normal load if anything is off.
RPC	Wraps <code>@server</code> functions: CSRF header, JSON, error translation into a catchable <code>ServerError</code> , redirects, optional page refresh.
Dev tools	Live reload over server-sent events, plus an on-page error overlay.

For comparison: React plus ReactDOM is roughly 140 KB before you add a router, a data layer or a build pipeline. The entire Jongo bundle for the example app — runtime plus every compiled component — is 57 KB uncompressed, served as one file with no toolchain.

Server functions

The bridge that removes your API layer. File: `jongo/rpc.py`.

```
@server(login_required=True)
def rename(request, todo: Todo, title: str) -> dict:
    todo.title = title
    todo.save()
    return todo.to_dict()
```

In a component you write `await rename(todo_id, "New title")`. Between those two lines Jongo does the work you would otherwise write by hand:

1. POSTs JSON to a generated endpoint, with the CSRF token attached.
2. Rejects the call if the user isn't logged in (or isn't an admin).
3. Binds the arguments to your signature and checks each one against its type hint.
4. Turns `todo: Todo` into a loaded row, or a 404 if it doesn't exist.
5. Serialises the result, converting models, dates and decimals to JSON.
6. Raises a catchable `ServerError` in the browser on failure, carrying field errors.

SECURITY BY CONSTRUCTION

- Only decorated functions are reachable. There is no generic "call any function" endpoint.
- Every unsafe request needs a CSRF token; cross-origin requests are refused.
- Arguments are validated before your code runs, so `title: str` can't arrive as a dict.
- Text is HTML-escaped on both sides; `raw()` is the one explicit escape hatch.
- `User.to_dict()` drops the password hash, so it can't leak into a page's props.

Compare with Django REST Framework. The same feature there is a serializer class, a viewset, a router entry, and a `fetch()` in JavaScript that duplicates the argument names. That's four places to keep in sync, in two languages, and the type checker can't see across the gap. Here the signature is the contract.

Models and migrations

Familiar querying, and schema changes without a directory of migration files. Files: `jongo/db/` (2,258 lines).

```
class Book(db.Model):
    title = db.Text(max_length=200)
    author = db.ForeignKey(Author, related_name="books")
    published = db.Date(null=True)

Book.filter(author__name__icontains="le guin", published__gte=date(1970, 1, 1))[:10]
Book.filter(db.Q(title__startswith="The") | db.Q(tags__contains="classic")).count()
author.books.create(title="The Dispossessed")
```

If you know Django's ORM you already know this one: managers, chainable lazy querysets, `__` lookups that traverse relations, `Q` objects, transactions with savepoints.

Migrations without files

DJANGO	JONGO
Two commands: <code>makemigrations</code> , then <code>migrate</code>	One: <code>jongo migrate</code>
A numbered file per change, committed to git	No files. The live schema is read back and compared to your models
Merge conflicts between branches that both add migrations	Nothing to conflict over
Destructive edits happen because a file said so	Dropping a column requires <code>--allow-destructive</code> ; safe changes apply on their own in dev
<code>sqlmigrate</code> to preview	<code>jongo migrate --plan</code> prints the exact SQL first

It creates tables, adds columns and indexes, and rebuilds a table when a column's type or nullability changes, copying the data across. Running it twice does nothing the second time.

The honest trade-off. Migration files are also a *history*, and a place to put data migrations ("split this column into two"). Jongo's approach is better for the common case of moving fast on one database, and weaker when you need a reviewed, replayable schema history across many environments. Right now it also only speaks SQLite.

Batteries: auth, admin, dev server

Auth

A `User` model with PBKDF2-SHA256 password hashing, `authenticate`, `login`, `logout`, and `request.user` everywhere — pages, routes and server functions. Sessions are HMAC-signed cookies, so there's no session table to clean up. Changing a password invalidates that user's other sessions, which is a nice property Django needs extra care to get.

The admin

Like Django's: `app.admin()` mounts a site at `/admin` that lists, searches, sorts, creates, edits and deletes rows for every model, building each form from your field types — checkboxes for booleans, dropdowns for foreign keys and choices, date pickers for dates. Passwords get a proper "set password" field and never display the hash. Create the first account with `jongo createadmin`.

The dev server

- Save a file and the server restarts *and* the open browser tab reloads itself.
- Errors render a debug page with the traceback, your source in context, and the request.
- Compile errors appear at startup with file, line and a hint.
- Browser exceptions surface in an on-page overlay instead of only the console.
- Safe schema changes apply automatically as you edit your models.

Testing

```
client = app.test_client()
assert client.get("/").status == 200
todo = client.rpc(add_todo, "Write tests")      # full round trip, CSRF included
assert client.navigate("/todos")["title"] == "Todos" # what the router would fetch
```

Jongo vs Django, honestly

Where Jongo is better

AREA	DJANGO	JONGO
Frontend	Not its problem. You add a template language, then JavaScript, then a bundler.	Written in Python, compiled to JS, rendered on the server first.
Feature layout	Split across 4–6 files by type.	One file, one feature.
Calling the server	DRF serializer + viewset + URL + fetch.	<code>await your_function(...)</code>
Validation	Form or serializer classes that restate the model.	Type hints on the function.
Migrations	Generated files, merge conflicts.	Diff against the live schema; destructive steps opt-in.
Build step	collectstatic, npm, a bundler, source maps.	None. One generated JS file.
Dependencies	Django plus whatever the frontend needs.	Zero. Standard library only.
Page transitions	Full reloads unless you add HTMX or a SPA.	Built-in: fetch the tree, patch the DOM, keep layout state.
Type safety across the wire	Impossible: Python one side, JS the other.	One language, one signature, checked at the boundary.
Learning surface	Python + templates + JS + a frontend framework.	Python.

Where Django is still better

AREA	REALITY
Maturity	Twenty years of production use and security review. Jongo is two days old and version 0.1.
Databases	PostgreSQL, MySQL, Oracle, plus connection pooling. Jongo: SQLite only.
Ecosystem	Thousands of packages — payments, CMS, search, social auth. Jongo has none.
Scale features	Caching layers, async views, channels/WebSockets, i18n, time zones, permissions and groups.
Migration history	Reviewable, replayable, with data migrations.
Hiring and answers	Every question has a Stack Overflow answer; Django developers are easy to find.

The fair summary: Jongo is better at the thing Django never tried to do — making the interactive frontend part of the same Python codebase — and it modernises a few backend habits along the way. Django remains the safer choice for a large team shipping a decade-long product on PostgreSQL. Jongo is the better choice for building something quickly, alone or in a small team, without the Python/JavaScript split.

By the numbers

6,662

lines of Python

1,476

lines of browser runtime

117

tests, all passing

0

dependencies

AREA	LINES	WHAT LIVES THERE
jongo/compiler/transpile.py	745	Python AST → JavaScript
jongo/compiler/pyrt.js	780	Python semantics in the browser
jongo/compiler/dom.js	696	Virtual DOM, hooks, router, RPC
jongo/db/	2,258	ORM, queries, fields, auto-migrations
jongo/vdom.py	515	The tree, server rendering, JSON bridge
jongo/app.py	492	Routing, pages, RPC endpoint, sessions, errors
jongo/admin.py	486	The generated admin site
jongo/http.py	399	Requests, responses, cookies, signed sessions
tests	1,860	ORM, compiler parity in Node, web layer, admin

Tested three ways: unit tests for the ORM; parity tests that run the same functions in Python and Node and demand identical answers; and end-to-end tests through an in-process HTTP client. On top of that, the example app was driven in a real Chrome browser — adding, toggling, filtering, deleting, navigating, and a timer with its cleanup — which is how three real bugs were found and fixed.

Limits, and what's next

KNOWN LIMITS TODAY

- **SQLite only.** PostgreSQL is the obvious next adapter.
- **No classes in browser code.** Components use functions, state and dicts.
- **No WebSockets.** Live updates today mean polling or a refresh after a server call.
- **No memoisation.** A state change re-renders that component's subtree; fine for pages of ordinary size, wasteful for very large lists.
- **One bundle.** Every component ships to every page; no code splitting yet.
- **Young.** Version 0.1, written in two days, not yet used in anger.

NATURAL NEXT STEPS

- A PostgreSQL adapter, then connection pooling.
- Per-page bundles, so a big app doesn't ship everything everywhere.
- WebSockets, so `state()` can follow the server.
- Sourcemaps back to the Python line, so browser stack traces point at your source.
- A component memo hook, and streaming server rendering.
- File uploads to storage backends, background jobs, an email helper.

Cheat sheet and file map

Commands

COMMAND	WHAT IT DOES
<code>jongo new mysite</code>	Create a project from the starter template
<code>jongo dev [--port 8000]</code>	Dev server: restarts on save, reloads the browser, debug pages
<code>jongo run --migrate</code>	Production server (or point gunicorn at <code>app:app</code>)
<code>jongo migrate --plan</code>	Show the SQL needed to match your models
<code>jongo createadmin</code>	Create an admin user
<code>jongo routes</code>	List every route
<code>jongo shell</code>	Python shell with your models loaded
<code>jongo build</code>	Compile all components and fail on errors — for CI

Where to look in VS Code

FILE	READ IT FOR
<code>examples/todo/app.py</code>	The whole idea, in one file. Start here.
<code>jongo/vdom.py</code>	What a component is, hooks, server rendering, the JSON bridge
<code>jongo/app.py</code>	A request's journey: routing → page → HTML, plus the RPC endpoint
<code>jongo/compiler/transpile.py</code>	The Python-to-JavaScript translation, statement by statement
<code>jongo/compiler/bundle.py</code>	How a name becomes an RPC stub, a component or an error
<code>jongo/compiler/dom.js</code>	Hydration, diffing, hooks and the router in the browser
<code>jongo/rpc.py</code>	Argument validation and the type-hint rules
<code>jongo/db/models.py</code>	The model metaclass and field descriptors
<code>jongo/db/migrate.py</code>	Schema diffing and table rebuilds
<code>tests/test_compiler.py</code>	The Python/JavaScript parity suite — a good spec of the supported subset